

Ciência dos Materiais

Física dos materiais

Prof. Dr. Diego J. Raposo

Escola Politécnica de Pernambuco – Universidade de Pernambuco



O que é o LAMMPS?

LAMMPS é um programa de simulação atomística comandado por arquivos de texto.

- O usuário define partículas, caixa, interações e operação desejada.
- O mesmo programa pode realizar dinâmica molecular, minimização, Monte Carlo por comandos específicos e vários métodos híbridos.
- A ordem das linhas importa: por exemplo, `units` e `atom_style` devem preceder a criação da caixa.
- Os nomes criados pelo usuário, como `box`, `relax` e `a0`, são identificadores.

Como executar no terminal (Linux)

```
lmp -in in.argon_gas.lmp  
ou, em paralelo:  
mpirun -np 4 lmp -in in.argon_gas.lmp
```

Estrutura do input

O input será organizado nas partes usuais de um arquivo LAMMPS:

- 1 Inicialização;
- 2 Criação do sistema;
- 3 Interações;
- 4 Lista de vizinhos;
- 5 Saídas;
- 6 Parâmetros da simulação;
- 7 Execução;
- 8 Salvamento da configuração final.

O input é uma sequência de instruções. Cada linha deve ser lida como:

comando	argumentos obrigatórios	opções
---------	-------------------------	--------

1. Inicialização: trecho do código

```
# ----- Initialization -----  
  
units          real  
atom_style     atomic  
dimension      3  
boundary       p p p
```

A inicialização define propriedades globais da simulação antes da criação dos átomos.

- units: sistema de unidades;
- atom_style: quais propriedades cada partícula carrega;
- dimension: dimensionalidade da simulação;
- boundary: condições de contorno.

1. Inicialização: unidades em real

Em `units real`, as unidades mais usadas são:

Grandeza	Unidade	Comentário
Comprimento	Å	coordenadas, σ , cutoff
Tempo	fs	timestep e tempos de amortecimento
Energia	kcal/mol	energia potencial e cinética
Massa	g/mol	massas atômicas
Temperatura	K	temperatura termodinâmica
Pressão	atm	pressão instantânea ou média
Densidade	g/cm ³	saída density

Essa escolha é apropriada quando queremos comparar diretamente com parâmetros experimentais ou de campos de força químicos.

1. Inicialização: exemplos de `atom_style`

<code>atom_style</code>	Sistema típico	O que adiciona
<code>atomic</code>	Ar, Ne, metais simples	massa, posição, velocidade
<code>charge</code>	saís, plasmas, íons	carga elétrica
<code>molecular</code>	moléculas sem carga explícita	topologia molecular
<code>full</code>	água, orgânicos, biomoléculas	carga + bonds + angles + dihedrals
<code>sphere</code>	partículas coloidais	raio e rotação

Para argônio monoatômico, `atomic` é suficiente: não há ligações, ângulos, diedros ou cargas.

2. Criando o sistema: trecho do código

```
# ----- Create system -----  
  
region          box block 0 200 0 200 0 200  
create_box      1 box  
  
create_atoms    1 random 1000 12345 box
```

Esse bloco define a caixa e cria os átomos.

- `region:` cria uma região geométrica chamada `box`;
- `block 0 200 0 200 0 200:` caixa cúbica de lado 200 Å;
- `create\_box 1 box:` permite 1 tipo de átomo nessa caixa;
- `create\_atoms 1 random 1000 12345 box:` cria 1000 átomos do tipo 1 em posições aleatórias.

2. Criando o sistema: tamanho e número de partículas

Neste exemplo:

$$L_x = L_y = L_z = 200 \text{ \AA}$$

$$V = L^3 = 200^3 = 8,0 \times 10^6 \text{ \AA}^3$$

$$N = 1000$$

A densidade numérica é:

$$\rho_N = \frac{N}{V} = \frac{1000}{8,0 \times 10^6} = 1,25 \times 10^{-4} \text{ átomos/\AA}^3$$

Essa densidade é baixa, portanto o sistema se comporta como gás diluído e as sobreposições iniciais tornam-se improváveis.

2. Criando o sistema: boas práticas

- Para gases, posições aleatórias costumam funcionar bem se a densidade for baixa.
- Para líquidos e sólidos, evite posições aleatórias densas: use uma rede inicial ou um arquivo data equilibrado (Ex.: Packmol).
- Sistemas muito pequenos apresentam flutuações grandes; para testes iniciais, $N \sim 10^3$ é razoável.
- Para propriedades estatísticas mais confiáveis, use mais partículas e tempos de simulação maiores.
- A menor dimensão da caixa deve ser significativamente maior que o raio de corte, para evitar artefatos geométricos.

Neste caso, $L = 200 \text{ \AA}$ e $r_c = 12 \text{ \AA}$, logo a caixa é muito maior que o alcance do potencial.

3. Interações: trecho do código

```
# ----- Interactions -----  
  
mass          1 39.948  
  
pair_style     lj/cut 12.0  
pair_modify    shift yes  
pair_coeff     1 1 0.2381 3.405
```

Esse bloco define a massa e o potencial entre as partículas.

- `mass 1 39.948`: massa molar do tipo 1, em g/mol;
- `pair_style lj/cut 12.0`: Lennard-Jones com cutoff de 12 Å;
- `pair_coeff 1 1 0.2381 3.405`: ϵ e σ para Ar-Ar;
- `pair_modify shift yes`: desloca o potencial para zerar no cutoff.

3. Interações: massa e potencial de Lennard-Jones

A massa entra diretamente na dinâmica, pois:

$$F_i = m_i a_i$$

A velocidade térmica típica depende de m :

$$\langle K \rangle = \frac{3}{2} k_B T$$

O potencial de Lennard-Jones é:

$$U(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

- ϵ : escala de energia da interação;
- σ : distância em que $U(r) = 0$;
- termo r^{-12} : repulsão de curto alcance;
- termo r^{-6} : atração dispersiva.

3. Interações: efeito de `pair_modify shift yes`

Com cutoff, o potencial seria truncado em r_c :

$$U_{\text{trunc}}(r) = \begin{cases} U(r), & r < r_c \\ 0, & r \geq r_c \end{cases}$$

Isso cria uma descontinuidade se $U(r_c) \neq 0$.

Com `pair_modify shift yes`, o LAMMPS usa:

$$U_{\text{shift}}(r) = \begin{cases} U(r) - U(r_c), & r < r_c \\ 0, & r \geq r_c \end{cases}$$

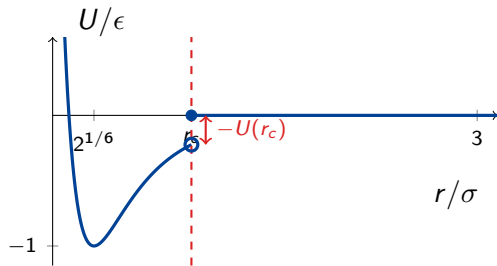
Assim:

$$U_{\text{shift}}(r_c) = 0$$

A energia fica contínua no cutoff, embora a força ainda possa ter descontinuidade se não for usado um potencial suavizado.

3. Interações: representação gráfica do shift

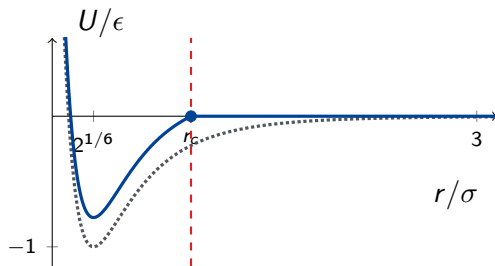
Potencial apenas truncado



Em r_c , o potencial apresenta um salto até zero.

Exemplo ilustrativo com $r_c = 1,6\sigma$: o shift soma a constante $-U(r_c)$, tornando $U_{\text{shift}}(r_c) = 0$.

Potencial deslocado: shift yes



— $U(r) - U(r_c)$ $U(r)$ original

4. Lista de vizinhos: trecho do código

```
# ----- Neighbor list -----  
  
neighbor          2.0 bin  
neigh_modify      delay 0 every 1 check yes
```

A lista de vizinhos evita calcular todos os pares de partículas em todos os passos.

$$\text{pares possíveis} \sim \frac{N(N-1)}{2}$$

Em vez disso, o LAMMPS calcula interações apenas para pares separados por menos que:

$$r_c + \text{skin}$$

Neste exemplo:

$$r_c = 12 \text{ \AA}, \quad \text{skin} = 2 \text{ \AA}$$

4. Lista de vizinhos: boas práticas

- neighbor 2.0 bin: usa uma região extra de $2 \text{ \AA}$ além do cutoff.
- bin: método eficiente para sistemas aproximadamente homogêneos.
- delay 0: permite reconstruir a lista desde o primeiro passo.
- every 1: verifica a lista a cada passo.
- check yes: reconstrói apenas se necessário, reduzindo custo computacional.

Boas práticas:

- skin pequeno demais aumenta reconstruções frequentes;
- skin grande demais aumenta pares desnecessários;
- em fases densas, monitore *dangerous builds*, que indicam que a lista de vizinhos pode estar sendo reconstruída tarde: átomos deslocaram-se bastante entre verificações, criando risco de pares entrarem no cutoff sem constarem na lista e de interações serem temporariamente ignoradas. Em fases condensadas, deve-se reconstruir a lista com maior frequência ou aumentar o skin.
- para gases diluídos, o custo de vizinhos tende a ser menor.

5. Saídas I: thermo_style e variáveis

```
# ----- Outputs -----  
  
thermo_style    custom step temp press density pe ke etotal vol  
thermo         100  
  
variable        vstep equal step  
variable        vtemp equal temp  
variable        vpress equal press  
variable        vdens equal density  
variable        vpe equal pe  
variable        vke equal ke  
variable        vetot equal etotal  
variable        vvol equal vol
```

thermo_style custom escolhe quais propriedades aparecem na tela e no log.lammps.
thermo 100 imprime essas grandezas a cada 100 passos.

5. Saídas I: propriedades termodinâmicas comuns

Tag	Propriedade	Unidade em real
step	passo da simulação	adimensional
temp	temperatura	K
press	pressão	atm
density	densidade de massa	g/cm ³
pe	energia potencial	kcal/mol
ke	energia cinética	kcal/mol
etotal	energia total	kcal/mol
vol	volume	Å ³
enthalpy	entalpia	kcal/mol
lx,ly,lz	lados da caixa	Å

As variáveis `vstep`, `vtemp`, etc. transformam propriedades termodinâmicas em variáveis *equal-style*, que podem ser usadas por comandos como `fix print`.

5. Saídas I: outras tags termodinâmicas úteis

Família	Tags comuns
Componentes energéticos	evdwl, ecoul, elong, ebond, eangle, edihed
Caixa	lx, ly, lz, xlo, xhi, xy, xz, yz
Tensão/pressão	pxx, pyy, pzz, pxy, pxz, pyz
Convergência	fmax, fnorm
Desempenho e vizinhos	cpu, spcpu, nbuild, ndanger
Valores personalizados	c_ID, f_ID, v_name

5. Saídas II: arquivo diagnostico.dat

```
fix                diag all print 100 &
"${vstep} ${vtemp} ${vpress} ${vdens} ${vpe} ${vke} ${vetot} ${vvol}" &
file diagnostico.dat &
screen no &
title "step temp press density pe ke etotal vol"
```

Esse fix cria um arquivo limpo com dados tabulados.

- diag: nome do fix;
- all: grupo de átomos ao qual o fix se aplica;
- print 100: escreve a cada 100 passos;
- file diagnostico.dat: arquivo de saída;
- screen no: evita duplicar essa saída na tela;
- title: primeira linha do arquivo, usada como cabeçalho.

5. Saídas II: por que usar o cabeçalho?

O título:

```
step temp press density pe ke etotal vol
```

faz com que o arquivo tenha colunas nomeadas.

Isso facilita a leitura automática em Python, pois o programa pode identificar:

- qual coluna deve ser usada como eixo x ;
- quais colunas podem ser usadas como eixo y ;
- os nomes corretos para legendas e rótulos;
- a diferença entre temperatura, pressão, energia e volume.

É mais limpo ler `diagnostico.dat` do que extrair manualmente dados do `log.lammps`.

5. Saídas III: trajetória

```
dump          1 all custom 1000 traj.lammpstrj id type x y z
dump_modify   1 sort id
```

O comando `dump` salva configurações microscópicas.

- 1: identificador do dump;
- all: salva todos os átomos;
- custom: escolhe manualmente as colunas;
- 1000: salva a cada 1000 passos;
- traj.lammpstrj: arquivo de trajetória;
- id type x y z: colunas salvas.

`dump\_modify 1 sort id` mantém os átomos ordenados por id, facilitando visualização e análise posterior.

6. Parâmetros da simulação: minimização

```
# ----- Simulation parameters -----  
  
min_style      cg  
minimize       1.0e-6 1.0e-8 1000 10000
```

cg significa gradiente conjugado.

Ele procura reduzir a energia potencial antes da dinâmica molecular.

Sintaxe geral:

```
minimize etol ftol maxiter maxeval
```

- etol: tolerância de energia;
- ftol: tolerância de força;
- maxiter: número máximo de iterações;
- maxeval: número máximo de avaliações de força.

6. Parâmetros da simulação: timestep e velocidades

```
timestep      1.0  
velocity      all create 120.0 12345 mom yes rot no dist gaussian
```

Em units real, timestep 1.0 significa: $\Delta t = 1 \text{ fs}$

Para argônio com potencial LJ, esse é um valor seguro para uma primeira simulação.

O comando velocity cria velocidades iniciais:

- all: aplica a todos os átomos;
- create 120.0: temperatura inicial de 120 K;
- 12345: semente aleatória;
- mom yes: remove momento linear total;
- rot no: não faz o ajuste para zerar o momento angular total;
- dist gaussian: distribuição de Maxwell-Boltzmann.

6. Parâmetros da simulação: boas práticas para timestep

O timestep deve ser pequeno o suficiente para resolver o movimento mais rápido.

Regras práticas:

- Argônio LJ em `real`: $\Delta t \sim 1$ fs é conservador;
- moléculas com ligações envolvendo H: frequentemente 0,25 – 1 fs, se ligações forem flexíveis;
- moléculas rígidas ou com constraints podem aceitar timesteps maiores;
- se a energia explode ou aparecem *lost atoms*, reduza Δt .

Para aulas introdutórias, é melhor começar com timestep menor e aumentar apenas depois de verificar estabilidade.

6. Parâmetros da simulação: fixes

```
fix          1 all nvt temp 120.0 120.0 100.0  
fix          LinMom all momentum 50 linear 1 1 1 angular
```

O primeiro `fix` aplica o ensemble NVT: $N, V, T = \text{constantes}$

`temp 120.0 120.0 100.0` significa:

- temperatura inicial: 120 K;
- temperatura final: 120 K;
- $T_{\text{damp}} = 100 \text{ fs}$.

O segundo `fix` remove deriva do centro de massa (nas três direções, 1 1 1) a cada 50 passos e a rotação coletiva do sistema. A correção de momento evita *center-of-mass drift*. Ela é útil em sistemas pequenos, simulações longas ou quando ruído numérico acumula momento total.

O `angular` remove variações no momento angular total, que é mais natural para um agregado finito (nanopartícula por exemplo). Portanto não é necessário aqui.

6. Parâmetros da simulação: T_{damp} , P_{damp} e deriva

Boas práticas para termostato e barostato:

$$T_{\text{damp}} \approx 100\Delta t$$

Com $\Delta t = 1$ fs:

$$T_{\text{damp}} \approx 100 \text{ fs}$$

Se fosse NPT, uma regra usual seria:

$$P_{\text{damp}} \approx 1000\Delta t$$

ou aproximadamente:

$$P_{\text{damp}} \sim 10 T_{\text{damp}}$$

7. Rodando o cálculo

```
# ----- Run -----  
  
run_style      verlet  
  
run            50000
```

`run\_style verlet` usa o integrador padrão de dinâmica molecular clássica.

Outros estilos existem para casos específicos, por exemplo:

- `verlet`: escolha padrão para MD clássica;
- `respa`: múltiplos passos de tempo para forças separadas por escala;
- `min`: usado internamente em minimizações;
- estilos especializados podem depender de pacotes adicionais.

7. Rodando o cálculo: tempo físico simulado

Com:

$$\Delta t = 1 \text{ fs}$$

$$N_{\text{steps}} = 50000$$

O tempo físico total é:

$$t = 50000 \times 1 \text{ fs} = 50000 \text{ fs}$$

$$t = 50 \text{ ps}$$

Para gases diluídos, esse tempo já permite observar estabilização de temperatura e flutuações termodinâmicas iniciais. Para propriedades de transporte ou estatísticas mais refinadas, seriam necessários tempos maiores.

8. Salvando a configuração final

```
# ----- Save final configuration -----  
  
write_data      final.data
```

O arquivo `final.data` salva uma configuração estrutural que pode conter:

- dimensões da caixa;
- número de átomos;
- tipos atômicos;
- massas;
- coordenadas finais;
- em sistemas moleculares, também topologia: bonds, angles, dihedrals etc.

Para reiniciar outra simulação a partir dessa estrutura, usa-se:

```
read_data      final.data
```

Input completo

```
# ----- Initialization -----
units          real
atom_style      atomic
dimension       3
boundary        p p p

# ----- Create system -----
region          box block 0 200 0 200 0 200
create_box      1 box
create_atoms    1 random 1000 12345 box

# ----- Interactions -----
mass            1 39.948
pair_style       lj/cut 12.0
pair_modify     shift yes
pair_coeff       1 1 0.2381 3.405

# ----- Neighbor list -----
neighbor        2.0 bin
neigh_modify    delay 0 every 1 check yes
```

Input completo: continuação

```
# ----- Outputs -----
thermo_style      custom step temp press density pe ke etotal vol
thermo            100

variable          vstep equal step
variable          vtemp equal temp
variable          vpress equal press
variable          vdens equal density
variable          vpe equal pe
variable          vke equal ke
variable          vetot equal etotal
variable          vvol equal vol

fix               diag all print 100 &
"${vstep} ${vtemp} ${vpress} ${vdens} ${vpe} ${vke} ${vetot} ${vvol}" &
file diagnostico.dat &
screen no &
title "step temp press density pe ke etotal vol"
```

Input completo: continuação

```
dump          1 all custom 1000 traj.lammpstrj id type x y z
dump_modify   1 sort id

# ----- Simulation parameters -----
min_style     cg
minimize      1.0e-6 1.0e-8 1000 10000

timestep      1.0
velocity      all create 120.0 12345 mom yes rot no dist gaussian

fix           1 all nvt temp 120.0 120.0 100.0
fix           LinMom all momentum 50 linear 1 1 1 angular

# ----- Run -----
run_style     verlet
run           50000

# ----- Save final configuration -----
write_data    final.data
```


Apêndices

O comando `units` escolhe um sistema completo de unidades

Estilo	Escalas principais	Uso típico
<code>lj</code> <code>real</code>	unidades reduzidas σ, ϵ , massa $\text{\AA}$, fs, kcal/mol, atm	modelos genéricos de fluidos e sólidos LJ química molecular, campos de força clássicos
<code>metal</code>	$\text{\AA}$, ps, eV, bar	metais, semicondutores, EAM/MEAM/SW/Tersoff
<code>si</code>	m, s, J, Pa, kg	problemas em unidades SI
<code>cgs</code>	cm, s, erg, barye	aplicações históricas em CGS
<code>electron</code>	bohr, fs, hartree	dinâmica de partículas eletrônicas
<code>micro</code>	μm , μs , pg	coloides e escalas micrométricas
<code>nano</code>	nm, ns, ag	sistemas mesoscópicos nanométricos

Fonte: Documentação oficial: docs.lammps.org/units.html.

Neste input, `units` real fixa todas as unidades

`units` `real`

Grandeza	Unidade	Onde aparece
comprimento	Å	a_0, σ, r_c, L_x
tempo	fs	dinâmica molecular, se houver
energia	kcal/mol	$p_e, E_{\text{coh}}, \epsilon$
massa	g/mol	mass
temperatura	K	temp
pressão	atm	press, box/relax
densidade	g/cm ³	density

Atenção

Mudar apenas a palavra `real` para `metal` sem converter os números altera o modelo físico.

O `atom_style` define quais dados cada átomo carrega

Estilo	Dados adicionais	Aplicação típica
<code>atomic</code>	conjunto mínimo	Ar, metais e sólidos monoatômicos
<code>charge</code>	carga q	saís, plasmas, íons sem topologia
<code>bond</code>	molécula + ligações	polímeros flexíveis simples
<code>angle</code>	ligações + ângulos	polímeros com rigidez angular
<code>molecular</code>	topologia molecular completa, sem q	moléculas apolares sem cargas explícitas
<code>full</code>	topologia + carga	água, orgânicos, biomoléculas, líquidos iônicos
<code>dipole</code>	carga + dipolo	partículas dipolares
<code>spin</code>	momento magnético	modelos spin-rede

Fonte: Documentação oficial: docs.lammps.org/atom_style.html.

Há estilos especializados para partículas não pontuais

Família	Estilos	Exemplos de uso
Forma e rotação	sphere, ellipsoid, line, tri, body	grãos, coloides, corpos rígidos, partículas anisotrópicas
Mesoscópicos	dpd, edpd, mdpd, tdpd, sph	DPD, hidrodinâmica de partículas, transporte de espécies
Sólidos contínuos	peri, smd, bpm/sphere	peridynamics, mecânica sólida, partículas ligadas
Campos específicos	amoeba, oxdna, electron, dielectric, apip	campos polarizáveis, DNA coarse-grained, elétrons, potenciais adaptativos
Composição	hybrid, template	união de atributos ou moldes moleculares

Regra prática: escolha o estilo mais simples que contenha os atributos exigidos; estilos especializados podem depender de pacotes compilados.

Lista de vizinhos: ideia central

Em potenciais de curto alcance, não é eficiente testar todos os pares de átomos a cada passo.

O LAMMPS constrói uma **lista de vizinhos** contendo os pares que estão próximos o suficiente para poderem interagir.

Ideia essencial

A lista de vizinhos não contém apenas os pares dentro do cutoff do potencial. Ela contém também uma margem extra chamada **skin**.

Comando neighbor

Exemplo típico:

```
neighbor      2.0 bin  
neigh_modify  delay 0 every 1 check yes
```

Se o potencial tem raio de corte r_c , a lista é construída até

$$r_{\text{lista}} = r_c + r_{\text{skin}}$$

No exemplo:

$$r_{\text{skin}} = 2.0 \text{ \AA}$$

em unidades como metal ou real.

O skin não é o cutoff da força

O **cutoff** define até onde a força de curto alcance é calculada.

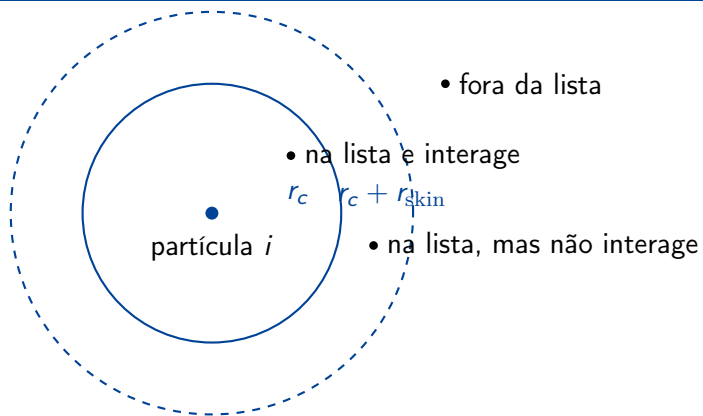
O **skin** define uma região extra na qual pares são guardados na lista, mas ainda não necessariamente interagem.

o skin controla quem fica de reserva na lista, não quem interage

Se $r_c = 10 \text{ \AA}$ e $r_{\text{skin}} = 2 \text{ \AA}$, então:

$$r_{\text{lista}} = 12 \text{ \AA}$$

Regiões ao redor de uma partícula



$0 < r < r_c$
na lista e interage

$r_c < r < r_c + r_{\text{skin}}$
na lista, mas não interage

$r > r_c + r_{\text{skin}}$
fora da lista

Partículas entram e saem da lista?

Entre duas reconstruções, a lista de vizinhos fica essencialmente fixa.

A cada passo, o LAMMPS verifica os pares que já estão armazenados:

- se o par está dentro de r_c , calcula a força;
- se está fora de r_c , a força de curto alcance não é calculada;
- o par pode continuar armazenado até a próxima reconstrução.

As entradas e saídas efetivas ocorrem quando a lista é reconstruída:

- pares dentro de $r_c + r_{\text{skin}}$ entram;
- pares fora de $r_c + r_{\text{skin}}$ saem.

Exemplo: par guardado no skin

Suponha:

$$r_c = 10 \text{ \AA}, \quad r_{\text{skin}} = 2 \text{ \AA}$$

Na construção da lista, um par está a

$$r_{ij} = 11.5 \text{ \AA}$$

Logo, ele está na lista, mas não interage naquele instante.

Depois de alguns passos, o par se aproxima:

$$r_{ij} = 9.8 \text{ \AA}$$

Agora ele passa a produzir força imediatamente, sem precisar esperar uma nova construção da lista, porque já estava armazenado graças ao skin.

Por que metade do skin?

Com

```
neigh_modify delay 0 every 1 check yes
```

LAMMPS verifica se algum átomo se deslocou demais desde a última construção.

Tipicamente, a reconstrução é disparada quando um átomo se desloca mais que

$$\frac{r_{\text{skin}}}{2}$$

Motivo: dois átomos podem se aproximar simultaneamente.

$$\Delta r_{\text{relativo}} \approx \Delta r_i + \Delta r_j$$

Se cada um anda metade do skin em sentidos opostos, a distância relativa pode diminuir aproximadamente um skin inteiro.

Interpretação geométrica

Um par inicialmente fora da lista satisfaz

$$r_{ij} > r_c + r_{\text{skin}}$$

Para esse par entrar na região de força, ele precisaria alcançar

$$r_{ij} < r_c$$

Portanto, a aproximação relativa teria que ser maior que a largura do skin.

Antes disso ocorrer, o deslocamento dos átomos normalmente dispara a reconstrução da lista.

Mensagem física

O skin evita que pares que se aproximam rapidamente entrem no cutoff sem estarem presentes na lista de vizinhos.

Skin pequeno

Um skin pequeno, por exemplo

$$r_{\text{skin}} = 0.2 \text{ \AA},$$

produz uma lista com poucos pares extras.

Vantagem:

- cada cálculo de força percorre uma lista menor;
- há menos pares que estão na lista sem interagir.

Desvantagem:

- pequenos deslocamentos já exigem reconstrução;
- a lista pode ser reconstruída muitas vezes;
- isso aumenta o custo computacional.

Um skin grande, por exemplo

$$r_{\text{skin}} = 5 \text{ \AA},$$

permite que os átomos se movam bastante antes de reconstruir a lista.

Vantagem:

- menos reconstruções da lista de vizinhos.

Desvantagem:

- muitos pares que não interagem ficam armazenados;
- o cálculo de força percorre uma lista maior;
- aumenta o uso de memória.

Interpretando o conjunto de comandos

```
neighbor      2.0 bin  
neigh_modify  delay 0 every 1 check yes
```

- neighbor 2.0 bin: usa skin de 2 unidades de comprimento e método espacial por caixas.
- delay 0: não impõe espera mínima antes de permitir reconstrução.
- every 1: a possibilidade de reconstrução é examinada a cada passo.
- check yes: a reconstrução só ocorre quando o deslocamento dos átomos indica necessidade.

Essa combinação é uma escolha segura para muitos tutoriais, pois monitora a lista frequentemente, mas evita reconstruções desnecessárias.

- Skin pequeno demais aumenta reconstruções frequentes.
- Skin grande demais aumenta o número de pares desnecessários.
- Em fases densas ou temperaturas altas, monitore os *dangerous builds*.
- Em gases diluídos, o custo de vizinhos tende a ser menor.
- Em simulações instáveis, cuidado: átomos podem se mover demais entre reconstruções.

Resumo

O valor ideal do skin é um compromisso entre custo de reconstrução e tamanho da lista de vizinhos.